

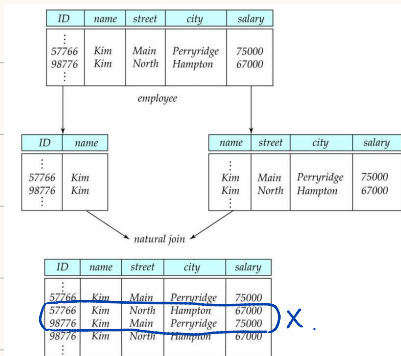
Ch8 Relational Database Design

8.1 Combined / Smaller Schmas

A Lossy Decomposition

有损分解(错误)

why: 分解中丢失了
依赖关系

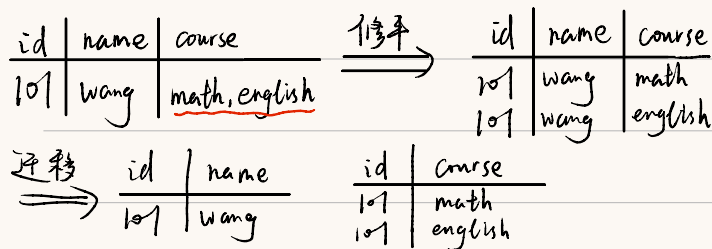


8.2 First Normal Form

(不允许多值)

Domain is atomic if its elements considered to be indivisible units.

R is in 1NF if the domains of all attrs of R are atomic.



-Goal: whether a relation R is in "good" form?

if not, decompose it into a set of relations $(R_1, \dots, R_n)$

- theory is based on $\begin{cases} \text{functional dependencies (FD)} \\ \text{multivalued dependencies (MVD)} \end{cases}$

★★★重点

8.3 Decomposition Using Functional Dependencies

$A \xrightarrow{\text{决定}} C$ $A \xrightarrow{1..1} F \xrightarrow{1..*} C$ A-样 C-样
自变 因变 即A, C有1:*关系 C-样 A不一定一样

- 函数依赖可表达 SK无法表示的约束

`inst_dept(ID, name, salary, dept_name, building, budget).`

We expect these functional dependencies to hold:

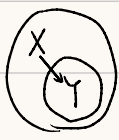
$dept_name \rightarrow building$

and $ID \rightarrow building$

but would not expect the following to hold:

$dept_name \rightarrow salary$

- Definitions

① Trivial FD 平凡依赖  $Y \subseteq X$
则必有 $X \rightarrow Y$

② Partial FD 部分函数依赖

对 $\alpha \rightarrow \beta$, 有 $\gamma \subset \alpha$, $\gamma \rightarrow \beta$

↓
partial (α 有冗余, β 不完全依赖于 α 中每个属性)

③ Transitive FD 传递依赖

$\alpha \rightarrow \beta, \beta \rightarrow \gamma \Rightarrow \alpha \rightarrow \gamma$
↓
transitive

闭包

- Closure of a set of FD

F 的闭包 $F^+ = \{f \mid F \text{ 蕴涵 } f\}$

e.g. $F = \{A \rightarrow B, B \rightarrow C\}$,

$F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C, \dots\}$

o Boyce-Codd Normal Form (BCNF)

设 F 为 R 中 FD 的集合, $\forall X \rightarrow Y \in F^+$

($X, Y \subseteq R$, F^+ 为 F 的闭包)

至少满足: ① $Y \subseteq X$ (即 $X \rightarrow Y$ is trivial)

or ② X is a SK of R

不可能有 CK 子集
决定其他属性

则 R 满足 BCNF

BCNF 无冗余依赖, 消除了冗余
(只允许整个超键决定其它属性)
保证了 R 中是必要信息

- Example schema not in BCNF:

① inst_dept (ID, name, dept_name, building)

($\because \text{dept_name} \rightarrow \text{building}$ and dept_name is not a SK)

② 存货 (仓库, 管理员, 商品名, 数量)

($\because \text{仓库} \rightarrow \text{管理员}$ and 仓库 is not a SK)

- problems: 1. 改 - 一个仓库管理员要应用到每个商品上
2. 插入异常: 无法给没有商品的仓库分配管理员
(仓库 $\rightarrow$ 管理员但商品这个主属性为空就不可插入)
3. 删除异常: 删除所有商品的仓库也天)

- Decomposing a schema into BCNF: ~~star~~ building $\rightarrow$ dept-name

Suppose we have non-trivial FD: $\alpha \rightarrow \beta$ and α isn't a SK

- ① $T = \alpha \cup \beta$: 形成新关系 (dept (dept_name, building))
- ② $R = R - \beta$: 去掉 α 决定的属性 (inst_dept (ID, name, dept_name))

- BCNF and Dependency Preservation:

dept_inst (stu_id, inst_id, dept_name)

CK: (stu_id, inst_id)

一个学生在一个系里只能有一个导师

FD: inst_id $\rightarrow$ dept_name

stu_id, dept_name $\rightarrow$ inst_id

根据上述规则分解后, we get:

(stu_id, inst_id) (inst_id, dept_name) 依赖丢失

we say our design is **not** dependency preserving

因为并非总能实现 BCNF 与 依赖保持, 故引入较弱的范式
(3NF)

Third Normal Form (3NF)

设 F 为关系 R 中所有 FD 的集合, $\forall X \rightarrow Y \in F^+$

($X, Y \subseteq R$, F^+ 为 F 闭包)

至少满足其一: ① $Y \subseteq X$

② X is a SK of R

③ $\forall a \in Y - X, a \in CK$ of R (即 a 是主属性)

则 R 满足 3NF

对于 $a \in X$
传递
(即 A 依赖)

(Note: each attribute may be in a different CK)

意义: $X \rightarrow Y$ 而 X 又非 SK, 就限制 $Y - X$ 的属性在 CK 中, 避免非主属性对 CK 部分依赖与恶性传递依赖

3NF 有一点冗余, 因为它允许部分依赖

允许主属性 (CK 的部分) 决定其它主属性

即在 BCNF 之外, 允许了主属性间的部分、传递依赖

避免了:

恶性传递

主属性 (CK 的一部分) $\xrightarrow{\text{对 CK 部分依赖}}$ 非主属性 $\longrightarrow$ 非主属性 $\longrightarrow$ 非主属性

只能:

被消除

主属性 (CK 的一部分) (非 SK) $\xrightarrow{\text{对 CK 部分依赖}}$ 主属性 (CK 的一部分) $\longrightarrow$ 主属性 (CK 的一部分) $\not\rightarrow$ 非主

• Second Normal Form (2NF)

设 F 为关系 R 中所有 FD 的集合, $\forall X \rightarrow Y \in F^+$

至少满足其一: ① ~ ③ 3NF 的条件

④ $X \subsetneq$ any CK of R

则 R 满足 2NF

↓
若 $X \neq SK$, $Y-X$ 也不是主属性
则起码 X 不能是 CK 的一部分

允许非主属性对 CK 的传递依赖
但不要非主属性对 CK 的部分依赖

• Summary

- 2NF

- **Def.2.** 如果关系模式 R 是 1NF, 而且每一个非主属性都完全依赖于 R 的主/候选键, 则 R 为 2NF

1NF $\xrightarrow{\text{消除非主属性对键的部分函数依赖}}$ 2NF

- 3NF

- **Def.2.** 如果关系模式 R 是 2NF, 而且每一个非主属性都不传递依赖于 R 的任何候选键, 则 R 是 3NF

2NF $\xrightarrow{\text{消除非主属性对键的传递函数依赖}}$ 3NF

- BCNF

- **Def.2.** 设 R 是 1NF, 如果对 R 中的每个属性 (包括主属性、非主属性), 不存在对键的传递或部分依赖, 则 R 是 BCNF。

3NF $\xrightarrow{\text{消除主属性对键的部分和传递函数依赖}}$ BCNF

8.4 functional Dependency Theory ★重点

• Armstrong's Axioms 公理

1. reflexivity 自反: $Y \subseteq X \Rightarrow X \rightarrow Y$
2. augmentation 增广: $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
3. transitivity 传递: $X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$

- 推论

1. union 合并: $X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$
($\because$ ① $XZ \rightarrow YZ$ ② $X \rightarrow Z \Rightarrow XX \rightarrow XZ$ 即 $X \rightarrow XZ \therefore X \rightarrow YZ$)
2. decomposition 分解: $X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$
3. pseudo-transitivity 伪传递: $X \rightarrow Y, YW \rightarrow Z \Rightarrow XW \rightarrow Z$
($\because XW \rightarrow YW$)

• Closure of a Set of FDs

Example: $R=ABC, F=\{A \rightarrow B, B \rightarrow C\}$, 求 F^+

$F^+ = \{A \rightarrow A, AB \rightarrow A, AC \rightarrow A, ABC \rightarrow A, B \rightarrow B, C \rightarrow C, A \rightarrow B, AB \rightarrow B, AC \rightarrow B, ABC \rightarrow B, B \rightarrow C, A \rightarrow C, AB \rightarrow C, AC \rightarrow C, ABC \rightarrow C, B \rightarrow BC, A \rightarrow AB, AB \rightarrow AB, AC \rightarrow AB, ABC \rightarrow AB, BC \rightarrow \Phi, A \rightarrow AC, AB \rightarrow AC, AC \rightarrow AC, ABC \rightarrow AC, BC \rightarrow B, A \rightarrow BC, AB \rightarrow BC, AC \rightarrow BC, ABC \rightarrow BC, BC \rightarrow C, A \rightarrow ABC, AB \rightarrow ABC, AC \rightarrow ABC, ABC \rightarrow A, BC \rightarrow BC\}$

① 用自反 & 增广滚雪球

② 用传递滚雪球

• Closure of Attribute Sets

$$\alpha^+ = \{ \beta \mid \alpha \rightarrow \beta \in F^+ \}$$

(由 α 通过 F 可以得到的所有属性的集合)

- an efficient algorithm to compute α^+ under F

Input: α , F Output: α^+

step 1. result = α ;

step 2. while (changes to result)

for each $\beta \rightarrow \gamma$ in F

if $\beta \subseteq \text{result}$ then result = result $\cup \gamma$;

when to end?
① when $\alpha^{(i+1)} = \alpha^{(i)}$
② when $\alpha^{(i)} = R$

复杂度 $\Rightarrow$

$O(|F| \times |U|)$

很高效

- uses of attribute closure

① testing for SK

α is SK $\Leftrightarrow \alpha^+ = R$.

② testing FDs

$\alpha \rightarrow \beta$ is in $F^+ \Leftrightarrow \beta \subseteq \alpha^+$

③ computing closure of F

for each $\gamma \subseteq R$, output $\gamma \rightarrow S$ ($S \subseteq \gamma^+$)

◦ Computing of Candidate Keys

定理: if $\forall \alpha \rightarrow \beta \in F$, $A \in \alpha$, then $A \in CK$
(只出现在左边的必为主属性)

算法过程:

Input: R, F Output: All CKs of R

step 1. 将 R 中属性分四类:

L类 R类 N类 LR类

令 $X = L \cup N$, $Y = LR$

step 2. if $X^+ = R$, then jump to step 5
else go to step 3

step 3. Y 中任取一属性 A

for each $A \in Y$

if $(X \cup A)^+ = R$

then $R = R \cup A$, jump to step 5

step 4. 否则, 从 Y 中取 2个、3个

直至 $(X \cup \alpha)^+ = R$, $X = X \cup \alpha$

} 时间复杂度可能达
 $O(2^n)$

step 5. end, output R

(NP-Hard)

◦ Extraneous Attributes

- Definition

① A is extraneous in α if $A \in \alpha$ 减少条件
and $F \Rightarrow (F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$

② A is extraneous in β if $A \in \beta$
and $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\} \Rightarrow F$ 减少结论

- Testing for extraneous attribute

① 若 $A \in \alpha$ (在左部)

1. compute $(\alpha - A)^+$ under F

2. if $\beta \subseteq (\alpha - A)^+$, then A is extraneous in α

② 若 $A \in \beta$ (在右部)

1. compute α^+ under $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$

2. if $A \in \alpha^+$, then A is extraneous in β

◦ Canonical Cover 正则覆盖

e.g. 冗余示例

① $\{A \rightarrow B, B \rightarrow C, \underline{A \rightarrow C}\}$ 冗余的函数依赖

- ② $\{A \rightarrow B, B \rightarrow C, A \rightarrow \textcircled{C}D\}$ 冗余的属性
 $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$

- Definition

F_c 是 F 的正则覆盖, 则:

- ① $F_c^+ = F^+$ (即 $F \Rightarrow F_c$ 且 $F_c \Rightarrow F$)
- ② F_c 中所有 FD 的左部是唯一的 (左部条件不重复)
- ③ F_c 中所有 FD 无冗余属性

- Compute a canonical cover for F

step 1. 用 union 法则同左合并: $\alpha_1 \rightarrow \beta_1, \alpha_1 \rightarrow \beta_2 \Rightarrow \alpha_1 \rightarrow \beta_1\beta_2$

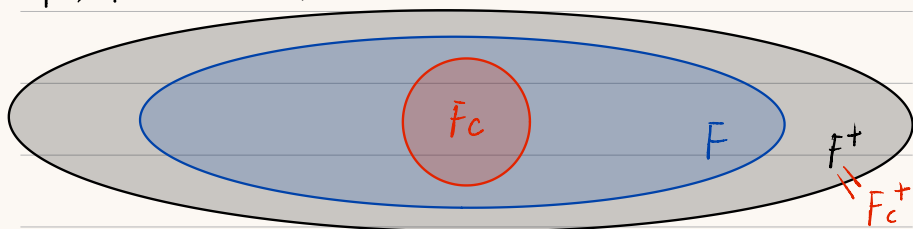
step 2. 删除冗余属性

step 3. 检查可否再合并

Attention:

冗余检验应用当前的函数依赖集 F_c 而非 F

- F, F^+ and F_c



• Minimum sets of FDs 最小函数依赖集

- Definition

F' 是 F 的最小集, 则:

① $F'^+ = F^+$

② F' 中所有 FD 右部必须为单属性

③ F' 中所有 FD 的左部无冗余属性 (右部单属性)

④ F' 中无冗余 FD

- 求解最小集算法

step 1. 用 decomposition 法则使所有 FD 右部为单属性

step 2. 去掉 F 中多余的依赖

for each $X \rightarrow A$ in F

compute X^+ under $F - \{X \rightarrow A\}$

if $A \in X^+$, then $X \rightarrow A$ is extraneous

step 3. 去掉各 FD 左部多余属性

- 注意: 最小集并不唯一, 与处理顺序有关

8.5 Problems in Decomposition ~~重点~~

• Lossless-join Decomposition

- definition

The decomposition $\{R_1, R_2, \dots, R_n\}$ of R is lossless decomposition if $r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \dots \bowtie \Pi_{R_n}(r)$
(if $r \neq \dots \Rightarrow$ lossy decomposition)

- sufficient condition

Suppose we have a decomposition on R into R_1 & R_2

$R_1 \cap R_2 \rightarrow R_1$ or $R_2 \Rightarrow$ this is a lossless decomposition

(also necessary if all constraints are FDs)

e.g. $R = (A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C\}$

decomposition ① $R_1 = (A, B)$ $R_2 = (B, C)$

$$R_1 \cap R_2 = B \rightarrow BC(R_2)$$

$$\textcircled{2} R_1 = (A, B) \quad R_2 = (A, C)$$

$$R_1 \cap R_2 = A \rightarrow ABC(R_1, R_2)$$

$\therefore$ ①, ② are both lossless-join decomposition

However, we cannot check $B \rightarrow C$ after decomposition ② without computing $R_1 \bowtie R_2$ in ②

$\therefore$ ② is not dependency preserving (无损不保依)

◦ Dependency Preservation

※ FD集的投影

F 是属性集 U 上的 FD 集, $Z \subseteq U$

$$\Pi_Z(F) = \{X \rightarrow Y \mid X \rightarrow Y \in F^+, X, Y \subseteq Z\}$$

- definition

The decomposition $\{R_1, R_2, \dots, R_n\}$ of R is dependency preserving if $F^+ = (\Pi_{R_1}(F) \cup \Pi_{R_2}(F) \cdots \cup \Pi_{R_n}(F))^+$

- testing for dependency preservation

for each $\alpha \rightarrow \beta \in F$

result = α

while (changes to result)

for each R_i in the decomposition

FD的传递性可能被分散到局部
逐次遍历无法推导出所有可
确定的属性, 需要反复迭代
直至 result 不再变化

$$t = \underbrace{(\text{result} \cap R_i)^+}_{(\text{under } F)} \cap R_i$$

$$\text{result} = \text{result} \cup t$$

if $\beta \subseteq \text{result}$, then the FD $\alpha \rightarrow \beta$ is preserved
else the decomposition isn't dependency preserving

• BCNF

think: why not compute CKs
and verify that $CK \subseteq \alpha$?

- simplified test

because compute CK may in $O(2^n)$

for each non-trivial FD $\alpha \rightarrow \beta$ in F rather than F^+

compute α^+ and verify that $\alpha^+ = R$

(α is a SK)

- testing decomposition for BCNF

① for each non-trivial FD $\alpha \rightarrow \beta$ in $\Pi_{R_i}(F^+)$

compute α^+ and verify that $\alpha^+ = R_i$

or

② for $\alpha \subseteq R_i$

compute α^+ (under F) and

verify that $\alpha^+ \cap (R_i - \alpha) = \emptyset$ or $\alpha^+ = R_i$

α 在 R_i 中无非平凡 FD

α 为 R_i 的 SK

- BCNF Decomposition Algorithm

result = $\emptyset$, remain = $\{R\}$

while (remain $\neq \emptyset$)

$S \in \text{remain}$, remain = remain - $\{S\}$

if (S isn't in BCNF)

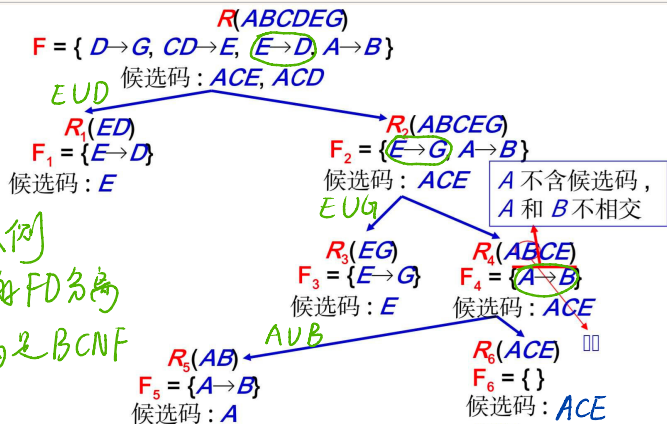
find $\alpha \rightarrow \beta$ that violates BCNF

let $R_i = \alpha\beta$, $S = (S - R_i) \cup \alpha$

remain = remain $\cup \{R_i, S\}$

else result = result $\cup \{S\}$

e.g.



每次都选双例
 均为矛盾的FD分解
 故 $\alpha \cup \beta$ 必满足 BCNF

$\rho = \{ R_1(ED), R_3(EG), R_5(AB), R_6(ACE) \}$
 $\Pi_{R_i}(F): \{E \rightarrow D\} \{E \rightarrow G\} \{A \rightarrow B\} \{\}$
 候选码: $E \quad E \quad A \quad ACE$

◦ 3NF

- testing for 3NF

Similar to simplified test for BCNF. However, when α isn't a SK ($\alpha^+ \neq R$), we need verify that each attribute in β is contained in a CK of R .

- 3NF Decomposition Algorithm

step 1. compute a canonical cover F_c for F

step 2. $i = 0$ // R_0 is null

for each FD $\alpha \rightarrow \beta \in F_c$

if $(\alpha \cup \beta) \not\subseteq R_j, \forall 1 \leq j \leq i$

$R_{i+1} = \alpha\beta$

} 正则覆盖中每个 FD 转换为一个新关系

step 3. if every CK of $R \not\subseteq R_j, \forall 1 \leq j \leq i$

$R_{i+1} = \text{any CK of } R$

} 补充 CK

step 4. for each schema $R_j, 1 \leq j \leq i$

if $R_j \subseteq R_k, 1 \leq k \leq i$

delete $R_j, i--$

} 删除多余关系

step 5. return $(R_1, R_2, \dots, R_i)$

e.g. $R(SNO, SD, Head, CNO, G)$

$F = \{SNO \rightarrow SD, SD \rightarrow Head, SNO \rightarrow Head, \{SNO, CNO\} \rightarrow G\}$

$CK = \{SNO, CNO\}$

① R is not in 3NF because Head is partially dependent on CK (see in transitive FD: $SNO \rightarrow Head$)

② Decomposition

Step1. $F_c = \{SNO \rightarrow SD, SD \rightarrow Head, (SNO, CNO) \rightarrow G\}$

Step2.

$R_1 = \{SNO, SD\}, F1 = \{SNO \rightarrow SD\}$

$R_2 = \{SD, Head\}, F2 = \{SD \rightarrow Head\}$

$R_3 = \{SNO, CNO, G\}, F3 = \{(SNO, CNO) \rightarrow G\}$

Step3. candidate key $\{SNO, CNO\}$ has been contained in R_3

Little Summary

判定范式的方法

给定关系模式和 FD 集，判断关系模式所属范式
解题步骤：

- 1) 求候选码，确定主属性和非主属性；
- 2) 看是否有非平凡 FD，其左部不含候选码；
- 3) 若不存在，则是 BCNF；
- 4) 否则：若这些 FD 的右部都是主属性，则是 3NF；
- 5) 否则：若任何候选码的任何真子集都不确定非主属性，则是 2NF；
- 6) 否则是 1NF。

例： $R(ABCD), F=\{AB \rightarrow C, A \rightarrow D\}$,
候选码：AB；主属性：A、B；非主属性：C、D
 $A \rightarrow D$ 的左部不含候选码，R 不是 BCNF
 $A \rightarrow D$ 的右部含非主属性，R 不是 3NF
 $A \rightarrow D$ ：候选码的真子集确定非主属性，R 不是 2NF
因此，R 是 1NF

Design Goals

根据是否保持依赖、是否无损分解将分解分成四类：

无损分解	保持依赖	说明
YES	YES	最好（不丢失数据和依赖）
YES	NO	可接受（丢失依赖，会导致异
NO	YES	不能接受（丢失数据）
NO	NO	不能接受（丢失数据和依赖）

问题：如何在保证无损和保持依赖的前提下，使分解所得的关系模式集符合尽可能高的范式？

目前有三个算法：

1. 保持依赖且无损地分解成 3NF 关系模式集
2. 无损地分解成 BCNF 关系模式集
3. 无损地分解成 4NF 关系模式集